Sclera AI Call Agent

Technical Documentation

Automated AI-powered voice calling system for technician notification and real-time availability confirmation — no human operator required

FastAPI | Twilio Voice API | OpenAI GPT-4o Realtime

WebSocket | Server-Sent Events | G.711 μ-law Audio

Where AI Is Used

Area	Description
Conversational AI	GPT-4o Realtime API conducts the full phone conversation
Speech-to-Text (STT)	OpenAI Realtime API transcribes technician's voice in real time
Text-to-Speech (TTS)	OpenAI Realtime API synthesizes the AI's spoken responses
Function Calling	AI calls end_call(status) to extract structured outcomes
Dynamic Prompting	System prompts generated per call with technician name and alarm data
Voice Activity Detection	Server-side VAD determines when the user has finished speaking

Table of Contents

1 Project Overview

2 AI Features

2.1 Real-Time Voice Conversation

2.2 Dynamic System Prompt Generation

2.3 AI Function Calling — Status Extraction

2.4 Real-Time Status Streaming via SSE

3 System Architecture

4 Implementation Details

5 Improvements & Future Enhancements

1 · Project Overview

What is this project?

Sclera AI Call Agent is an automated, AI-powered voice calling system. It places outbound phone calls to technicians on behalf of the Sclera platform, conducts a natural voice conversation using a real-time AI assistant, and captures the technician's response — all without any human operator.

What problem does it solve?

When an alarm or incident is detected in the Sclera platform, a technician must be notified and their availability must be confirmed. Doing this manually is slow and resource-intensive. This system automates the entire process: it calls the technician, verbally describes the issue, asks if they can handle it, and returns a structured response (accepted or rejected) to the backend — in real time.

2 · AI Features

Feature 1: Real-Time Voice Conversation

Purpose

The core AI feature. Connects a live phone call to OpenAI's GPT-4o Realtime API and conducts a natural, spoken conversation with the technician. The AI greets the technician by name, describes the detected alarm, asks if they are available to handle it, and wraps up the call — all in real-time, bidirectional audio.

Endpoint

```
WebSocket: /{request_id}/media-stream
(Opened by Twilio when technician answers)
```

Workflow

1. Backend calls POST /twilio-call-agent with alarm data and technician info
2. Server initiates an outbound phone call via Twilio
3. Technician's phone rings; when answered, Twilio opens a WebSocket to /{request_id}/media-stream
4. Server accepts the WebSocket and connects to OpenAI Realtime API (wss://api.openai.com/v1/realtime)
5. OpenAI session created with instructions, voice settings, audio format, and end_call tool definition
6. OpenAI triggered to generate its opening greeting immediately
7. Task A (concurrent): receives audio from Twilio → forwards to OpenAI via input_audio_buffer.append
8. Task B (concurrent): receives AI audio from OpenAI (response.audio.delta) → forwards to Twilio
9. Server-side VAD detects when technician stops speaking (silence > 1500ms)
10. OpenAI processes audio input, generates response, streams audio back
11. Once conversation is complete, AI invokes the end_call function with a status (accepted/rejected)
12. Status is stored for the call completion handler

Technologies Used

Technology	Detail
Model	GPT-4o Realtime Preview (gpt-4o-realtime-preview-2025-06-03)
Protocol	WebSocket — bidirectional, persistent
Audio Codec	G.711 μ-law (8kHz) — compatible with standard telephone networks
Voice	shimmer (configurable via VOICE environment variable)
Concurrency	Python asyncio.gather for simultaneous send/receive

VAD & Audio Settings

Setting	Value	Purpose
VAD threshold	0.9	High sensitivity — avoids false triggers from background noise
Prefix padding	500ms	Captures speech that occurs before VAD activates
Silence duration	1500ms	Waits 1.5s of silence before ending a turn
Ping interval	5s	WebSocket keep-alive ping frequency
Ping timeout	10s	Connection considered dead if no pong received

Edge Cases Handled

- Non-English speaker → AI always responds in English only

- Alarm JSON has no active issues → AI informs the technician and asks if they need help
- Ambiguous response → AI re-engages conversationally before concluding
- OpenAI fails to connect → TwiML error message played to caller and call ends gracefully

Failure Handling

Connection timeout after 20 seconds. Up to 3 retries with exponential backoff (2^attempt seconds). HTTP 401 (invalid API key) → immediately aborts, no retries. Unexpected connection close → caught and retried.

Feature 2: Dynamic System Prompt Generation

Purpose

Generates a personalized, data-aware instruction set for the AI at the start of every call. Each prompt is unique — it contains the technician's actual name and the real alarm data for that call.

Workflow

1. get_system_message(request_id, twilio_call_requests_log) is called when a WebSocket session starts
2. Technician's name retrieved from the call log
3. Alarm JSON for that specific call is serialized to a string
4. Both are injected into a multi-step instruction template (7-step behavioral guide)
5. Resulting prompt sent to OpenAI as part of session.update

Prompt Design

- Includes step-by-step behavioral instructions: greeting, alarm analysis, status detection, and graceful termination
- Tone guidance embedded — AI directed to be warm, friendly, and conversational
- AI instructed not to repeat static phrases, keeping each call feeling natural
- Affirmative responses ('Sure', 'Absolutely', 'I can') → mapped to accepted
- Rejection responses ('Not now', 'I'm busy', 'I have a meeting') → mapped to rejected

Technologies Used

Python f-strings for dynamic prompt construction | json.dumps for alarm data serialization

Feature 3: AI Function Calling — Status Extraction

Purpose

Instead of parsing free-form conversation text to determine the outcome, the AI is given a formal tool (end_call) that it must call when the conversation is complete. This guarantees a clean, structured 'accepted' or 'rejected' response from every successful call.

Workflow

1. At session setup, the end_call tool is registered with OpenAI via FUNCTION_DEFINITIONS
2. tool_choice set to 'auto' — OpenAI decides when to call it
3. As conversation concludes, OpenAI invokes end_call({status: 'accepted'}) or end_call({status: 'rejected'})
4. OpenAI emits response.function_call_arguments.done OR response.output_item.done
5. process_status() handles both event types, parses arguments JSON, extracts status field
6. Status stored: twilio_call_requests_log[request_id]['openai_final_status'] = status
7. When Twilio sends 'completed' status callback, stored OpenAI status replaces it in the event stream

Edge Cases Handled

- Neither event fires before call ends → status defaults to 'incomplete'
- Malformed JSON arguments → caught by try/except, logged, call continues
- Invalid status values → silently ignored; only 'accepted' or 'rejected' are stored

Failure Handling

JSON parse failures logged as [Status Extraction Error]. Dual event-type handling ensures status is captured regardless of which OpenAI event fires first.

Feature 4: Real-Time Status Streaming via Server-Sent Events (SSE)

Purpose

Keeps the calling backend system informed of every stage of the call — in real time — without polling. Streams intermediate status updates (ringing, in-progress) and, at completion, the final AI-determined outcome plus the full call history.

Endpoints

```
POST /twilio-call-agent → returns StreamingResponse (text/event-stream)
POST /status-callback → Twilio webhook that feeds events into the stream
```

SSE Event Types

Event	When Sent	Key Fields
initiated	Call created	status, description, created_at
ringing	Phone ringing	status, description, created_at
in-progress	Technician answered	status, description, created_at
accepted / rejected / incomplete	Call ended	status, description, created_at
final	Stream closing	type: final, call_history object

Workflow

1. POST /twilio-call-agent creates unique request_id (UUID) and an asyncio.Queue
2. Twilio initiates the outbound call; StreamingResponse returned immediately
3. As call progresses, Twilio POSTs to /status-callback for each state change
4. Each state mapped to a human-readable description and queued as an SSE event
5. Intermediate states (initiated, ringing, in-progress): queued and streamed immediately
6. Terminal state (completed): final status event queued (with OpenAI's accepted/rejected substituted if available)
7. Full call history assembled and queued as a 'final' event
8. _STREAM_END sentinel queued to close the stream cleanly
9. Queue and log entry removed from memory in finally block

Technologies Used

FastAPI StreamingResponse — text/event-stream media type | asyncio.Queue — thread-safe event passing between webhook handler and SSE generator | Sentinel object pattern — clean stream termination without polling

Failure Handling

status_callback exceptions → caught, logged with traceback, returns HTTP 500. Queue not found for request_id → events silently dropped. Client disconnects → try/finally ensures cleanup.

3 · System Architecture

Overview

The system is a real-time telephony AI pipeline. Three services interact: the FastAPI Server (orchestrates everything), the Twilio Voice API (places the call and streams audio), and the OpenAI Realtime API (provides the conversational AI brain). The FastAPI server acts as the central hub, bridging Twilio's audio stream with OpenAI's intelligence and streaming results back to the originating backend.

Component Roles

Component	Role
FastAPI (routes.py)	API gateway — handles all HTTP and WebSocket connections
websocket_service.py	Orchestrates WebSocket session setup and audio bridging
openai_utils.py	Manages OpenAI Realtime API connection, session config, and status extraction
audio_utils.py	Handles bidirectional audio streaming between Twilio and OpenAI
making_call.py	Initiates outbound Twilio calls with TwiML and callback URLs
twilio_utils.py	Twilio client, phone number validation, call history assembly
prompt.py	Dynamic system prompt generation and function tool definitions

Call Lifecycle Flow

Phase	Who Acts	What Happens
1. Initiation	Backend → FastAPI	POST /twilio-call-agent with alarm and technician data
2. Call Placement	FastAPI → Twilio	client.calls.create() with TwiML and status_callback URLs
3. SSE Open	FastAPI → Backend	StreamingResponse returned; backend receives live event stream
4. Ringing	Twilio → FastAPI	POST /status-callback → SSE event streamed
5. Answer	Technician → Twilio	Call answered; Twilio opens WebSocket media-stream
6. AI Connect	FastAPI → OpenAI	WebSocket to wss://api.openai.com/v1/realtime opened
7. Session Setup	FastAPI → OpenAI	session.update with instructions, tools, VAD; response.create triggers greeting
8. Conversation	OpenAI ↔ Twilio	Bidirectional G.711 audio streamed; VAD manages turn-taking
9. Status Extract	OpenAI → FastAPI	end_call({status}) function call captured
10. Completion	Twilio → FastAPI → Backend	status-callback completed → SSE final events → stream closed

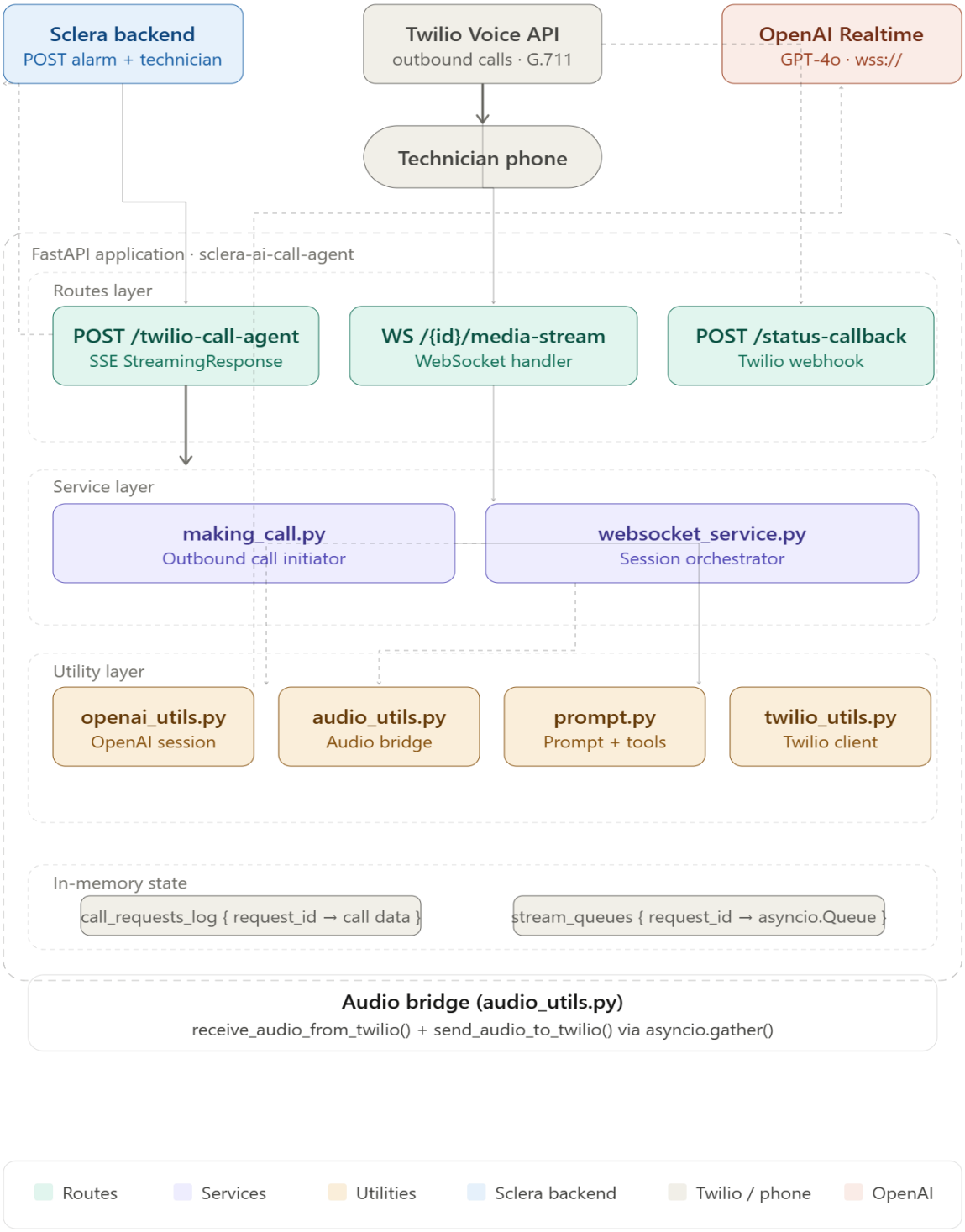
In-Memory Data Store

The application uses simple Python dictionaries for state. All state is transient (valid only for the duration of a call) and cleaned up immediately after each call ends.

Store	Key	Contents
twilio_call_requests_log		call_sid, alarm_json, technician_name, contact_number, technician_id, ai_call_log_id, vdms_id, call_status events, openai_final_status
request_stream_queues		asyncio.Queue — receives SSE events from Twilio webhook, consumed by SSE generator

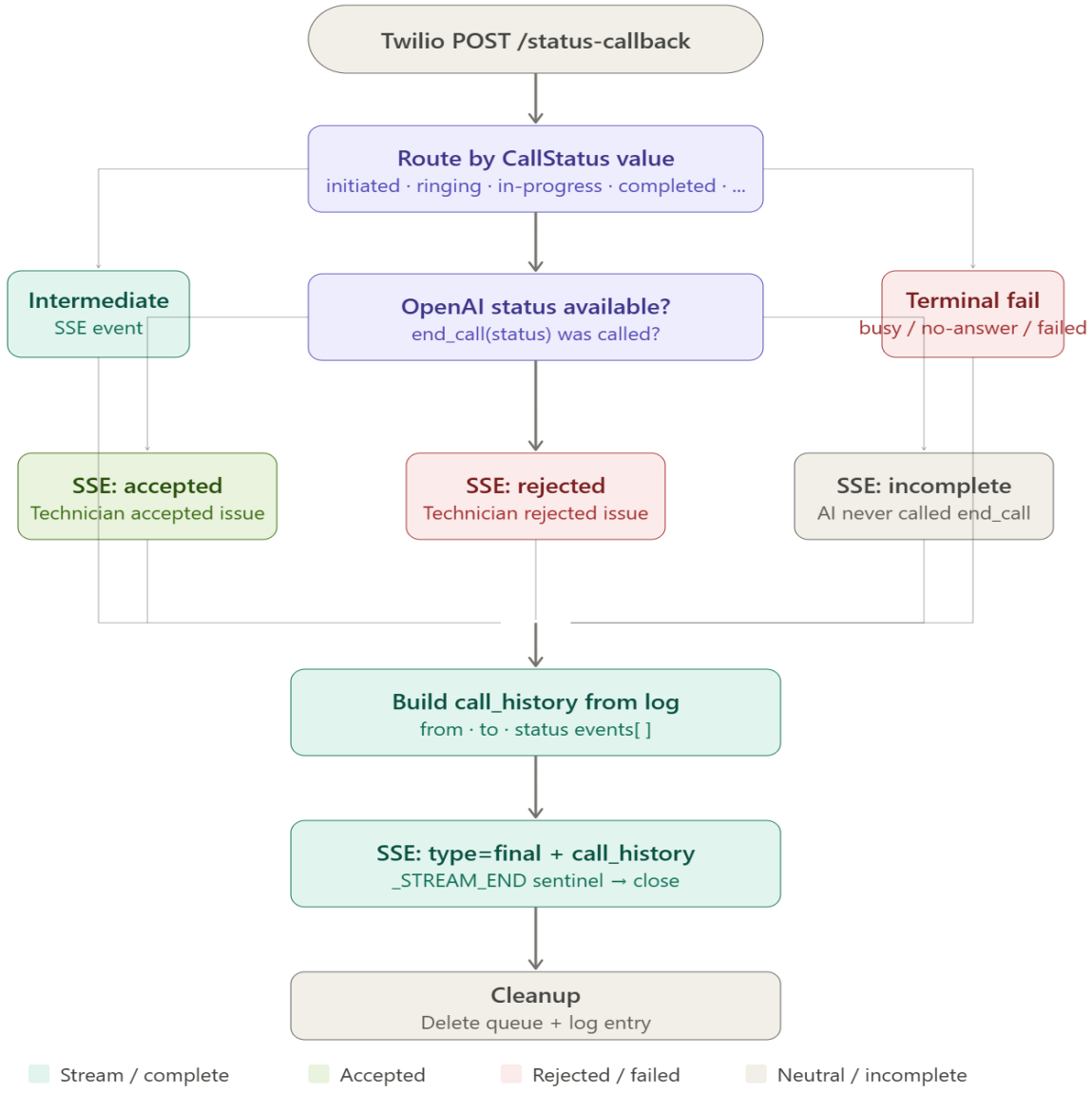
System Architecture Diagram

The diagram below shows the full application architecture — the three external services (Sclera backend, Twilio Voice API, OpenAI Realtime), the FastAPI routes layer, service layer, utility layer, and in-memory state stores.



Status Callback & SSE Flow Diagram

The diagram below illustrates how each Twilio status-callback event is routed — through the OpenAI status check — to one of the SSE outcomes (accepted, rejected, incomplete, or terminal fail), and then through call history assembly and stream cleanup.



4 · Implementation Details

Key Design Decisions

OpenAI Realtime API over Separate STT + TTS Services

A single OpenAI Realtime API connection handles STT, LLM reasoning, and TTS in one stream. This eliminates the latency of chaining three separate API calls and allows truly real-time, streaming audio responses.

G.711 μ-law Audio Codec

Twilio's media streams use G.711 μ-law encoding by default. The OpenAI session is configured to accept and output the same format, avoiding any audio transcoding — reducing latency and complexity.

Function Calling for Status Extraction

Instead of parsing the AI's final speech to extract intent, the AI is given a formal end_call(status) tool. This guarantees machine-readable, schema-validated, deterministic output ('accepted' or 'rejected') regardless of how the conversation flowed.

SSE for Real-Time Updates

A StreamingResponse is opened when the call is initiated and kept open until the call terminates, streaming each status change as it happens. One long-lived HTTP connection instead of many polling requests.

asyncio.Queue as Event Bus

The /status-callback webhook handler and the SSE generator run in different async contexts. An asyncio.Queue acts as a safe, decoupled event bus between them, avoiding race conditions.

UUID-based Request Tracking

Every call gets a unique request_id (UUID) generated at POST time. This UUID threads through all services — TwiML URLs, status callback URLs, WebSocket paths, and memory — ensuring events from different calls are never mixed up.

In-Memory State (No Database)

Simple Python dictionaries for state management. Since all state is transient (valid only for the duration of a call) and the app is single-process, a database would add unnecessary complexity. State is cleaned up immediately after each call.

Important Logic Patterns

Exponential Backoff for OpenAI Connection

```
Attempt 1: connect immediately
Attempt 2: wait 1 second (2^0)
Attempt 3: wait 2 seconds (2^1)
```

Status Resolution at Call Completion

When Twilio sends 'completed', the server checks if openai_final_status was set:

- If yes → replace 'completed' with 'accepted' or 'rejected'
- If no → status becomes 'incomplete' (call ended without AI extracting a decision)

Session Initialization Sequence

```
1. send session.update (VAD config, voice, tools, instructions)
2. receive session.created confirmation
3. send conversation.item.create (empty user message to prime context)
4. send response.create (triggers AI to generate opening greeting)
```

5 · Improvements & Future Enhancements

Performance Improvements

Improvement	Description
Horizontal scaling	Replace in-memory state with Redis to allow multiple server instances and load balancing
Connection pooling	Pre-warm OpenAI connections or use a connection pool to reduce first-response latency
Audio buffering	Add a small jitter buffer for audio packets to improve speech quality on unstable networks

Accuracy Improvements

Improvement	Description
Ambiguous response fallback	Implement a second prompt pass or escalation path when AI cannot determine accepted/rejected
Language detection	Detect caller's preferred language to brief the AI, even while responding in English
Alarm data enrichment	Pass richer context (asset type, severity, location) in the alarm JSON for more specific AI descriptions

Scaling Ideas

Improvement	Description
Persistent storage	Move call logs to PostgreSQL or MongoDB for auditing, analytics, and replay
Message queue	Replace Twilio webhook → asyncio.Queue with Redis Pub/Sub or RabbitMQ for resilience across restarts
Multi-tenant support	Add org-level API key management and routing for multiple Sclera customers
Call recording	Integrate Twilio call recording and store transcripts for compliance and quality review
Retry on no-answer	Automatically retry the call or escalate to another technician after a configurable delay

Developer Experience

Improvement	Description
Structured logging	Replace print() with structlog or Python logging (JSON format) for production observability
Health check endpoint	Add GET /health to validate Twilio and OpenAI credentials are reachable
Unit tests	Add pytest coverage for process_status, get_system_message, receive_call_history, and status callback handler
Environment validation	Add startup checks verifying all required env vars are present before accepting requests